Corporación Universitaria
Antonio José de Sucre

ÁRBOL AVL

Documentación y algoritmo

Presentado por:

Luis Gabriel Rivas Díaz

Robinson Andrés Quintero Lastra

Corporación Universitaria Antonio José de Sucre

Septiembre de 2026

Contenido

1. Introducción
2. Punto de partida: el árbol binario de búsqueda
3. ¿Qué es un árbol AVL?
4. Altura y factor de equilibrio
5. Estructura del nodo
6. Las rotaciones
7. Algoritmos: inserción, eliminación, búsqueda y recorridos
8. Ejemplo paso a paso
9. Análisis de complejidad
10. Comparación, ventajas y desventajas
11. Aplicaciones
12. Implementación en Python
13. Conclusiones
14. Referencias

1. Introducción

Las estructuras de datos jerárquicas permiten almacenar información de forma ordenada y recuperarla rápidamente. Entre ellas, el árbol binario de búsqueda (ABB) es una de las más conocidas, pero su rendimiento depende de la forma que adopte el árbol: si las claves llegan casi ordenadas, el árbol se convierte en una lista enlazada y las operaciones dejan de ser eficientes.

El árbol AVL resuelve este problema. Es un árbol binario de búsqueda que se equilibra automáticamente después de cada inserción o eliminación, de modo que su altura siempre crece de forma logarítmica respecto al número de elementos.

Este documento explica qué es un árbol AVL, cómo funciona, cuáles son sus algoritmos principales y cuál es su costo computacional. Además, se acompaña de una implementación completa en Python (archivo arbol_avl.py) y de una presentación.

2. Punto de partida: el árbol binario de búsqueda

Un árbol binario de búsqueda cumple una regla de ordenamiento: para cada nodo, todas las claves del subárbol izquierdo son **menores** y todas las del subárbol derecho son **mayores**. Esto permite buscar descartando la mitad del árbol en cada comparación, siempre que el árbol esté equilibrado.

El problema aparece cuando se insertan claves ya ordenadas (por ejemplo 10, 20, 30, 40, 50). Cada clave nueva se agrega como hijo derecho de la anterior y el árbol queda como una cadena de altura **n**. En ese caso buscar, insertar o eliminar cuesta **O(n)**, igual que recorrer una lista.

Un árbol AVL evita esa degeneración: después de cada modificación comprueba el equilibrio y, si hace falta, reorganiza los nodos mediante rotaciones.

3. ¿Qué es un árbol AVL?

El árbol AVL fue propuesto en 1962 por los matemáticos soviéticos **G. M. Adelson-Velsky** y **E. M. Landis**, de quienes toma su nombre. Fue la primera estructura de árbol binario de búsqueda autobalanceado que se conoció.

Definición

Un árbol AVL es un árbol binario de búsqueda en el que, para **todo** nodo, la diferencia entre la altura de su subárbol izquierdo y la de su subárbol derecho es como máximo 1 (en valor absoluto). Esta condición se llama **propiedad AVL**.

Propiedades

- Cumple todas las propiedades de un ABB: el recorrido inorden entrega las claves en orden ascendente.

- Todo subárbol de un AVL es también un AVL.
- Su altura es siempre O(log n): con n nodos, la altura h cumple $h < 1{,}44 \cdot \log_2(n + 2)$.
- Se rebalancea automáticamente con rotaciones, que tienen costo constante.

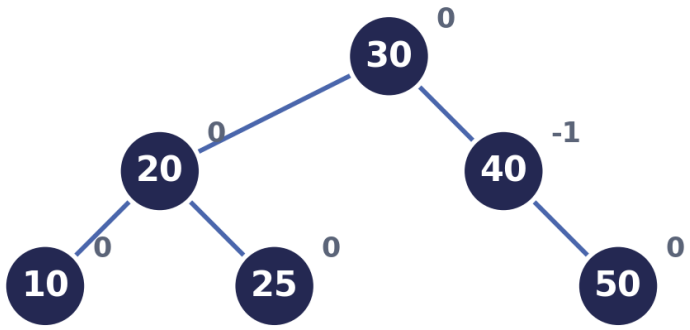


Figura 1. Ejemplo de árbol AVL. El número junto a cada nodo es su factor de equilibrio.

4. Altura y factor de equilibrio

La **altura** de un nodo es la cantidad de nodos del camino más largo hasta una hoja descendiente. En este documento un subárbol vacío tiene altura 0 y una hoja tiene altura 1, por lo que:

```
altura(nodo) = 1 + max( altura(nodo.izq), altura(nodo.der) )
```

El **factor de equilibrio (FE)** de un nodo mide qué tan inclinado está:

```
FE(nodo) = altura(nodo.izq) - altura(nodo.der)
```

FE	Estado	Significado
0	Equilibrado	Ambos subárboles tienen la misma altura.
+1	Aceptable	El subárbol izquierdo es un nivel más alto.
−1	Aceptable	El subárbol derecho es un nivel más alto.
+2 ó −2	Desbalanceado	Se viola la propiedad AVL: hay que rotar.

Un nodo puede llegar a FE = ±2 solamente después de una inserción o eliminación en uno de sus subárboles, porque antes de la operación el árbol era un AVL válido. Por eso basta con revisar los ancestros del nodo modificado.

5. Estructura del nodo

Cada nodo almacena la clave, las referencias a sus dos hijos y su altura. Guardar la altura evita recorrer el subárbol cada vez que se necesita el factor de equilibrio.

```
class Nodo:
    def __init__(self, clave):
        self.clave = clave
        self.izq = None      # hijo izquierdo
        self.der = None      # hijo derecho
        self.altura = 1      # una hoja tiene altura 1
```

6. Las rotaciones

Una rotación es una reorganización local de tres nodos que reduce la altura del subárbol desbalanceado sin alterar el orden de las claves. Solo se reasignan referencias, por lo que cada rotación cuesta O(1). Según dónde ocurra el desbalance existen cuatro casos:

Caso	Dónde crece el árbol	Condición	Solución
Izquierda-Izquierda	Subárbol izquierdo del hijo izquierdo	FE = +2 y FE(hijo izq.) ≥ 0	Rotación simple a la derecha
Derecha-Derecha	Subárbol derecho del hijo derecho	FE = −2 y FE(hijo der.) ≤ 0	Rotación simple a la izquierda
Izquierda-Derecha	Subárbol derecho del hijo izquierdo	FE = +2 y FE(hijo izq.) < 0	Rotación izquierda en el hijo y luego derecha en el nodo
Derecha-Izquierda	Subárbol izquierdo del hijo derecho	FE = −2 y FE(hijo der.) > 0	Rotación derecha en el hijo y luego izquierda en el nodo

6.1 Rotación simple a la derecha (caso Izquierda-Izquierda)

El hijo izquierdo del nodo desbalanceado se convierte en la nueva raíz del subárbol. El nodo desbalanceado pasa a ser su hijo derecho y el antiguo hijo derecho del hijo pasa a ser el hijo izquierdo del nodo desbalanceado.



Figura 2. Rotación simple a la derecha: el nodo 30 estaba desbalanceado.

```
ROTAR_DERECHA(y):
    x <- y.izq
    B <- x.der
    x.der <- y          // y baja a la derecha de x
    y.izq <- B          // el subárbol B cambia de padre
    actualizar altura(y); actualizar altura(x)
    retornar x          // nueva raíz del subárbol
```

6.2 Rotación simple a la izquierda (caso Derecha-Derecha)

Es la imagen en espejo de la anterior: el hijo derecho sube, el nodo desbalanceado baja a su izquierda y el antiguo hijo izquierdo del hijo pasa a ser el hijo derecho del nodo desbalanceado.

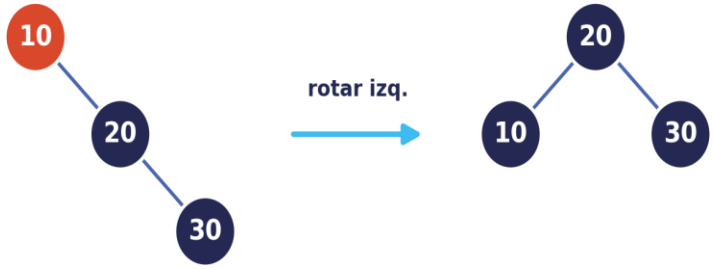


Figura 3. Rotación simple a la izquierda: el nodo 10 estaba desbalanceado.

6.3 Rotación doble izquierda-derecha (caso Izquierda-Derecha)

Cuando el hijo izquierdo está inclinado hacia la derecha, una sola rotación no basta. Primero se rota a la izquierda el hijo izquierdo, lo que deja el caso Izquierda-Izquierda, y luego se rota a la derecha el nodo desbalanceado.

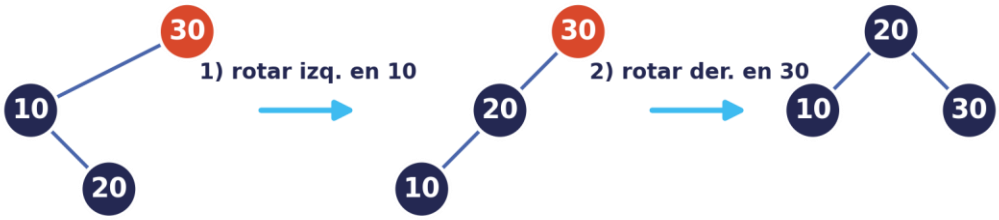


Figura 4. Rotación doble izquierda-derecha.

6.4 Rotación doble derecha-izquierda (caso Derecha-Izquierda)

Es el caso simétrico: primero se rota a la derecha el hijo derecho y luego se rota a la izquierda el nodo desbalanceado.

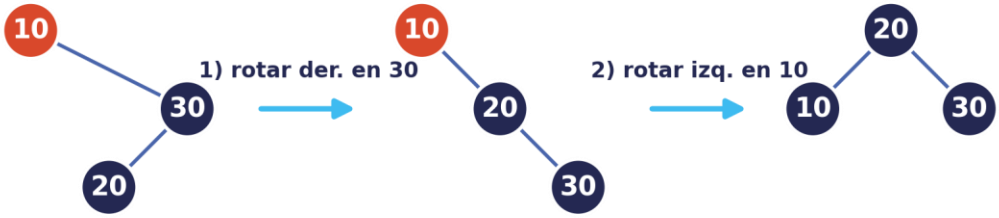


Figura 5. Rotación doble derecha-izquierda.

7. Algoritmos

7.1 Rebalanceo

Todas las operaciones que modifican el árbol usan el mismo procedimiento al regresar por la recursión: actualizar la altura del nodo, calcular su factor de equilibrio y aplicar la rotación que corresponda.

```
REBALANCEAR(nodo):
    actualizar altura(nodo)
    fe <- FE(nodo)
    si fe > 1:                              // pesa a la izquierda
        si FE(nodo.izq) < 0:                // caso Izquierda-Derecha
            nodo.izq <- ROTAR_IZQUIERDA(nodo.izq)
        retornar ROTAR_DERECHA(nodo)
    si fe < -1:                             // pesa a la derecha
        si FE(nodo.der) > 0:                // caso Derecha-Izquierda
            nodo.der <- ROTAR_DERECHA(nodo.der)
        retornar ROTAR_IZQUIERDA(nodo)
    retornar nodo                           // ya estaba balanceado
```

7.2 Inserción

1. Insertar la clave como en un ABB: comparar y bajar por la izquierda o la derecha hasta llegar a un lugar vacío. Si la clave ya existe, no se inserta.
2. Al regresar por la recursión, llamar a REBALANCEAR en cada ancestro.
3. En una inserción basta con una rotación (simple o doble) para restablecer el equilibrio de todo el árbol.

```
INSERTAR(nodo, clave):
    si nodo es nulo: retornar nuevo Nodo(clave)
    si clave < nodo.clave:  nodo.izq <- INSERTAR(nodo.izq, clave)
    sino si clave > nodo.clave: nodo.der <- INSERTAR(nodo.der, clave)
    sino: retornar nodo                     // clave repetida
    retornar REBALANCEAR(nodo)
```

7.3 Eliminación

1. Buscar el nodo como en un ABB.
2. Eliminarlo según su cantidad de hijos: sin hijos (hoja) se quita; con un hijo, ese hijo ocupa su lugar; con dos hijos, se copia la clave de su sucesor inorden (el mínimo del subárbol derecho) y se elimina ese sucesor.

3. Al regresar hacia la raíz, llamar a REBALANCEAR en cada ancestro.

A diferencia de la inserción, una eliminación puede requerir una rotación en cada nivel del camino de regreso, es decir, hasta O(log n) rotaciones.

```
ELIMINAR(nodo, clave):
    si nodo es nulo: retornar nulo
    si clave < nodo.clave:  nodo.izq <- ELIMINAR(nodo.izq, clave)
    sino si clave > nodo.clave: nodo.der <- ELIMINAR(nodo.der, clave)
    sino:
        si nodo.izq es nulo: retornar nodo.der
        si nodo.der es nulo: retornar nodo.izq
        s <- MINIMO(nodo.der)               // sucesor inorden
        nodo.clave <- s.clave
        nodo.der <- ELIMINAR(nodo.der, s.clave)
    retornar REBALANCEAR(nodo)
```

7.4 Búsqueda

Es idéntica a la de un ABB: se compara la clave con la del nodo actual y se baja a la izquierda o a la derecha hasta encontrarla o llegar a un nodo vacío. Como la altura es O(log n), la búsqueda también lo es.

7.5 Recorridos

Recorrido	Orden de visita	Uso típico
Inorden	Izquierda, raíz, derecha	Obtener las claves ordenadas.
Preorden	Raíz, izquierda, derecha	Copiar o guardar la estructura del árbol.
Postorden	Izquierda, derecha, raíz	Liberar o eliminar el árbol.
Por niveles	De arriba abajo, de izquierda a derecha	Ver el árbol nivel por nivel (usa una cola).

8. Ejemplo paso a paso

Se insertan en un AVL vacío las claves 10, 20, 30, 40, 50 y 25, en ese orden:

1. **Insertar 10 y 20.** No hay desbalance: 20 queda como hijo derecho de 10.
2. **Insertar 30.** El nodo 10 queda con FE = −2 (caso Derecha-Derecha). Se aplica una rotación simple a la izquierda y 20 pasa a ser la raíz.
3. **Insertar 40.** Se agrega como hijo derecho de 30. Todos los nodos siguen equilibrados.
4. **Insertar 50.** El nodo 30 queda con FE = −2 (caso Derecha-Derecha). Rotación simple a la izquierda: 40 sube y 30 y 50 quedan como sus hijos.
5. **Insertar 25.** Baja por 20 → 40 → 30 y queda como hijo izquierdo de 30. La raíz 20 queda con FE = −2 y su hijo derecho (40) con FE = +1: caso Derecha-Izquierda. Se rota a la derecha en 40 y luego a la izquierda en 20; la nueva raíz es 30.

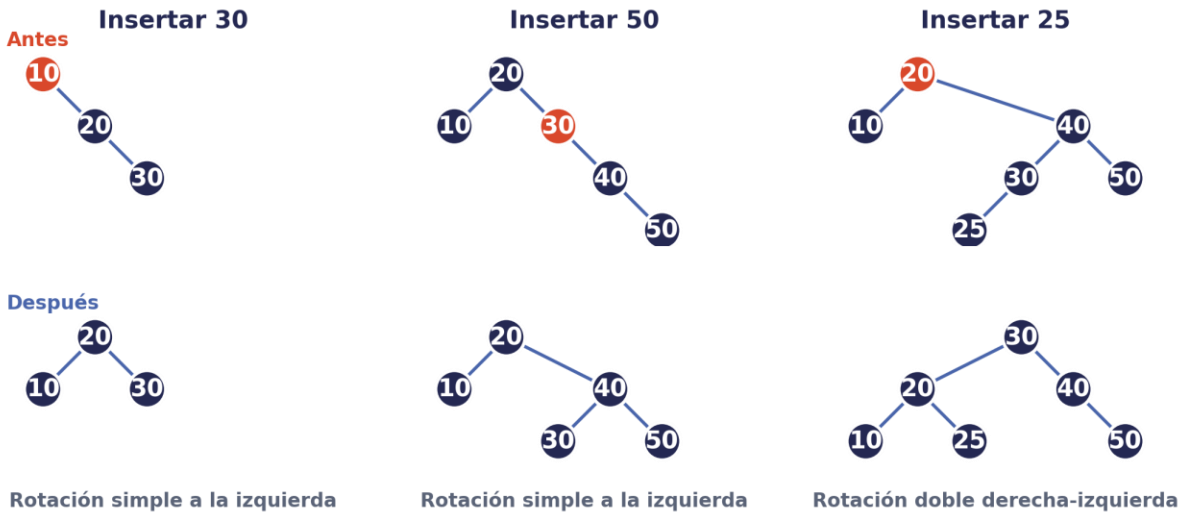


Figura 6. Inserciones que provocan rotación. El nodo rojo es el desbalanceado.

Recorridos del árbol final: inorden **[10, 20, 25, 30, 40, 50]**, preorden **[30, 20, 10, 25, 40, 50]**, postorden **[10, 25, 20, 50, 40, 30]** y por niveles **[30, 20, 40, 10, 25, 50]**. La altura final es 3.

9. Análisis de complejidad

9.1 Por qué la altura es logarítmica

Sea N(h) la cantidad mínima de nodos que puede tener un AVL de altura h. Para que el árbol sea lo más "flaco" posible, la raíz tiene un subárbol de altura h − 1 y otro de altura h − 2, ambos también mínimos:

```
N(h) = N(h-1) + N(h-2) + 1      con N(1) = 1  y  N(2) = 2
```

Esta recurrencia crece como la sucesión de Fibonacci, es decir, exponencialmente con h. Al despejar h se obtiene que un AVL con n nodos tiene altura menor que $1{,}44 \cdot \log_2(n + 2)$. Por tanto, la altura es O(log n).

9.2 Costos de las operaciones

Operación	Peor caso	Justificación
Búsqueda	O(log n)	El camino recorrido no supera la altura del árbol.
Inserción	O(log n)	Se baja y se sube una vez por el camino; a lo sumo una rotación.
Eliminación	O(log n)	Igual, pero puede haber una rotación por nivel.
Recorridos	O(n)	Cada nodo se visita exactamente una vez.
Espacio	O(n)	Un nodo por clave más el entero de la altura.

10. Comparación, ventajas y desventajas

Característica	ABB simple	Árbol AVL	Árbol rojo-negro
Balanceo	Ninguno	Estricto (|FE| ≤ 1)	Más flexible
Búsqueda (peor caso)	O(n)	O(log n)	O(log n)
Inserción (peor caso)	O(n)	O(log n)	O(log n)
Rotaciones al insertar	Ninguna	A lo sumo 1 (simple o doble)	A lo sumo 2
Mejor para	Datos aleatorios y pocos	Muchas consultas	Muchas modificaciones

Ventajas

- Búsqueda garantizada en O(log n): no se degenera como un ABB.
- Muy eficiente cuando hay muchas más consultas que modificaciones.
- Mantiene las claves ordenadas y permite recorrerlas en orden.

Desventajas

- Insertar y eliminar cuestan más por las rotaciones y el cálculo de alturas.
- Cada nodo guarda un dato adicional (altura o factor de equilibrio).
- La implementación es más compleja que la de un ABB, sobre todo la eliminación.
- Con muchas escrituras, un árbol rojo-negro suele hacer menos reorganizaciones.

11. Aplicaciones

- **Índices en memoria:** estructuras de búsqueda para bases de datos con predominio de lecturas.
- **Diccionarios y conjuntos ordenados:** cuando se necesitan las claves en orden además de buscarlas.
- **Sistemas con tiempo de respuesta predecible:** el peor caso de búsqueda está acotado por log n.
- **Formación académica:** es la base para comprender otros árboles balanceados, como el rojo-negro o el árbol B.

12. Implementación en Python

La implementación completa está en el archivo **arbol_avl.py**, que se entrega por separado. La clase **ArbolAVL** incluye las siguientes operaciones:

Método	Descripción
insertar(clave)	Inserta una clave y rebalancea. Devuelve False si ya existía.
eliminar(clave)	Elimina una clave y rebalancea. Devuelve False si no existía.

Método	Descripción
buscar(clave)	Devuelve True si la clave está en el árbol.
minimo(), maximo()	Devuelven la clave menor y la mayor.
altura()	Devuelve la altura del árbol.
inorden(), preorden(), postorden(), por_niveles()	Devuelven los recorridos como listas.
esta_balanceado()	Verifica que todos los nodos cumplan |FE| ≤ 1 y que las alturas guardadas sean correctas.
mostrar()	Dibuja el árbol en texto, con el factor de equilibrio de cada nodo.

Ejemplo de uso

```
from arbol_avl import ArbolAVL

arbol = ArbolAVL()
for clave in [10, 20, 30, 40, 50, 25]:
    arbol.insertar(clave)

print(arbol.inorden())     # [10, 20, 25, 30, 40, 50]
print(arbol.preorden())    # [30, 20, 10, 25, 40, 50]
print(arbol.altura())      # 3
print(arbol.buscar(25))    # True
arbol.eliminar(40)
print(arbol.esta_balanceado())    # True
```

Al ejecutar el archivo directamente se muestra una demostración paso a paso y se corre una prueba automática: 200 árboles aleatorios a los que se les insertan 100 claves y luego se les eliminan 50, verificando después de cada operación que el árbol siga siendo un AVL válido y que el recorrido inorden esté ordenado.

```
python arbol_avl.py
```

13. Conclusiones

- El árbol AVL es un árbol binario de búsqueda que se autobalancea usando el factor de equilibrio y rotaciones.
- Solo existen cuatro casos de desbalance, y cada uno se resuelve con una rotación simple o doble de costo constante.
- Su altura es siempre logarítmica, lo que garantiza O(log n) en búsqueda, inserción y eliminación, incluso en el peor caso.
- Es una buena elección cuando se prioriza una búsqueda rápida y predecible sobre la velocidad de las modificaciones.

14. Referencias

Adelson-Velsky, G. M. y Landis, E. M. (1962). An algorithm for the organization of information. Doklady Akademii Nauk SSSR, 146, 263–266.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. Introduction to Algorithms. MIT Press.

Knuth, D. E. The Art of Computer Programming, Vol. 3: Sorting and Searching. Addison-Wesley.

Weiss, M. A. Data Structures and Algorithm Analysis. Pearson.